INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS DISTRIBUIDOS

Práctica 6. Servicios Web

ALUMNO:

CRUZ SANTOS CRISTIAN OZIEL

PROFESOR:

CHADWICK CARRETO ARELLANO

GRUPO:

7CM5

FECHA: 15/04/2026

1. Antecedentes

Servicios Web y Arquitectura REST

La evolución de los sistemas distribuidos ha llevado a la estandarización de la comunicación mediante **Servicios Web**. A diferencia de los sockets crudos, que operan en niveles bajos de red, los servicios web utilizan el protocolo **HTTP** y formatos de intercambio de datos como **JSON**. La arquitectura REST (Representational State Transfer) permite que los sistemas interactúen mediante "recursos" identificables, facilitando la interoperabilidad entre diferentes plataformas y lenguajes.

Comunicación Asíncrona y Polling

En aplicaciones de alto rendimiento, es ineficiente mantener una conexión abierta mientras el servidor realiza una tarea larga. El modelo de comunicación asíncrona permite que el cliente envíe una petición y reciba una respuesta inmediata (un ID de seguimiento). Posteriormente, el cliente puede realizar consultas periódicas (**Polling**) mediante el método **GET** para verificar el avance de su tarea sin bloquear sus propios recursos de ejecución.

2. Problemática

Abstracción de Protocolos y Gestión de Estados

Migrar un sistema de comandas basado en sockets a un entorno web real implica resolver retos de diseño superiores:

- **Acoplamiento de Red:** Los sockets requieren que cliente y servidor gestionen manualmente la conexión y el formato de los bytes. El reto es subir el nivel de abstracción a la capa de aplicación (HTTP).
- **Inactividad Bloqueante:** En las prácticas anteriores, el mesero no podía realizar otras tareas mientras el chef cocinaba. Se requiere un mecanismo donde el mesero pueda "desconectarse" y preguntar después si el platillo está listo.
- **Persistencia de Información:** El servidor debe ser capaz de mantener un registro temporal de todos los pedidos activos y sus estados internos para responder a consultas externas en cualquier momento.

3. Propuesta de Solución

API REST de Cocina con Consultas de Estado

Se implementa un **Servicio Web** utilizando el framework Flask que expone tres funcionalidades principales:

- **Registro de Pedidos (Método POST):** El endpoint */pedido* recibe un JSON con la comanda. El servidor genera un identificador único (UUID), inicia el proceso de cocina en un hilo independiente (protegido por un **Semáforo de 3 estufas**) y responde de inmediato con el ID de seguimiento.
- **Consulta de Progreso (Método GET):** El endpoint */estatus/<id_pedido>* permite al cliente verificar en tiempo real si su orden está en estado: *En espera, Cocinando* o *Listo*.
- **Sincronización Híbrida:** Se mantiene el uso de semáforos contadores para asegurar que, aunque la API web sea accesible masivamente, la capacidad física de la cocina (recursos compartidos) sea respetada de forma disciplinada.

4. Implementación

Los códigos desarrollados se encuentran en el siguiente repositorio de Github:

https://github.com/CruzCris/Sistemas_Distribuidos_7CM1

5. Resultados

Escenario de Prueba:

Se desplegó el servicio web en el puerto 5000. Se ejecutó un cliente que realizó una petición de "Pizza Especial" y, tras recibir su ID, procedió a consultar el estatus cada 3 segundos.

Observaciones en Consola:

- **Desacoplamiento Exitoso:** El servidor devolvió el ID de pedido en milisegundos, liberando al cliente de la espera inmediata.
- **Monitoreo Dinámico:** Se documentó la transición de estados en el cliente: primero recibiendo "En espera de estufa", luego "Cocinando" (cuando el semáforo le otorgó el paso) y finalmente "Listo".
- **Interoperabilidad:** Se comprobó que el estatus también podía ser consultado desde un navegador web ingresando la URL del endpoint GET, demostrando la naturaleza pública y estandarizada del servicio.

```
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos\P6> py servidor.py
 * Serving Flask app 'servidor'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 903-921-432
```

Levantamiento del Servicio Web

```
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos\P6> py cliente.py
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos\P6> py cliente.py
[MESERO] Enviando comanda vía Web Service...
[MENSAJE] Tu ID de seguimiento es: a0e9e027
```

Cliente 1

```
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos> cd P6
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos\P6> py cliente.py
[MESERO] Enviando comanda vía Web Service...
[MENSAJE] Tu ID de seguimiento es: 3dc06d29
```

Cliente 2

```
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos> cd P6
PS D:\TAREAS\ESCOM\8vo\Sistemas Distribuidos\P6> py cliente.py
[MESERO] Enviando comanda vía Web Service...
[MENSAJE] Tu ID de seguimiento es: d894ba1b
```

Cliente 3

```
[CONSULTA] Estatus del pedido a0e9e027: Cocinando
[CONSULTA] Estatus del pedido a0e9e027: Cocinando
[CONSULTA] Estatus del pedido a0e9e027: Cocinando
[CONSULTA] Estatus del pedido a0e9e027: Listo
```

Seguimiento del cliente 1

```
[CONSULTA] Estatus del pedido 3dc06d29: Cocinando
[CONSULTA] Estatus del pedido 3dc06d29: Cocinando
[CONSULTA] Estatus del pedido 3dc06d29: Cocinando
[CONSULTA] Estatus del pedido 3dc06d29: Listo
```

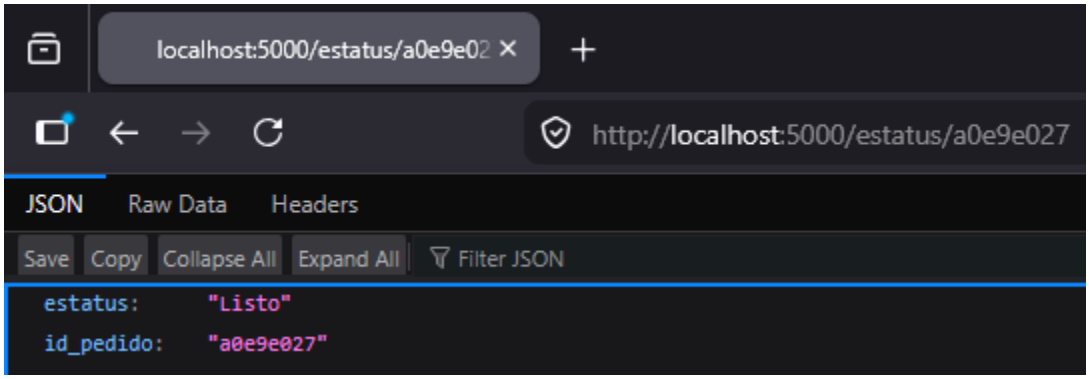
Seguimiento del cliente 2

```
[CONSULTA] Estatus del pedido d894ba1b: Cocinando
[CONSULTA] Estatus del pedido d894ba1b: Cocinando
[CONSULTA] Estatus del pedido d894ba1b: Cocinando
[CONSULTA] Estatus del pedido d894ba1b: Listo
```

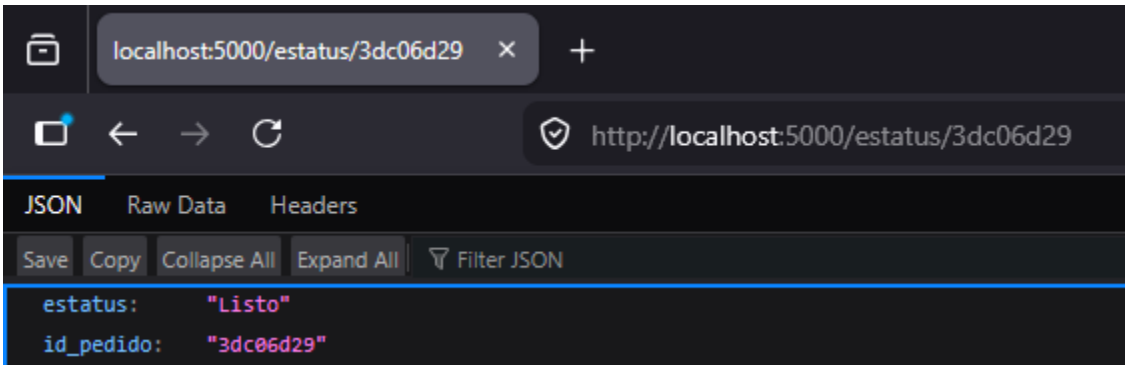
Seguimiento del cliente 3

```
127.0.0.1 - - [15/Apr/2026 22:22:45] "GET /estatus/123 HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:22:45] "GET /favicon.ico HTTP/1.1" 404 -
[API] Cocinando pedido a0e9e027: Pizza Especial ESCOM
127.0.0.1 - - [15/Apr/2026 22:22:56] "POST /pedido HTTP/1.1" 202 -
127.0.0.1 - - [15/Apr/2026 22:22:58] "GET /estatus/a0e9e027 HTTP/1.1" 200 -
[API] Cocinando pedido 3dc06d29: Pizza Especial ESCOM
127.0.0.1 - - [15/Apr/2026 22:22:59] "POST /pedido HTTP/1.1" 202 -
127.0.0.1 - - [15/Apr/2026 22:23:01] "GET /estatus/3dc06d29 HTTP/1.1" 200 -
[API] Cocinando pedido d894ba1b: Pizza Especial ESCOM
127.0.0.1 - - [15/Apr/2026 22:23:03] "POST /pedido HTTP/1.1" 202 -
127.0.0.1 - - [15/Apr/2026 22:23:03] "GET /estatus/a0e9e027 HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:23:05] "GET /estatus/d894ba1b HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:23:06] "GET /estatus/3dc06d29 HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:23:08] "GET /estatus/a0e9e027 HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:23:10] "GET /estatus/d894ba1b HTTP/1.1" 200 -
[API] Pedido a0e9e027 terminado.
127.0.0.1 - - [15/Apr/2026 22:23:11] "GET /estatus/3dc06d29 HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:23:13] "GET /estatus/a0e9e027 HTTP/1.1" 200 -
[API] Pedido 3dc06d29 terminado.
127.0.0.1 - - [15/Apr/2026 22:23:15] "GET /estatus/d894ba1b HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:23:16] "GET /estatus/3dc06d29 HTTP/1.1" 200 -
[API] Pedido d894ba1b terminado.
127.0.0.1 - - [15/Apr/2026 22:23:20] "GET /estatus/d894ba1b HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:24:18] "GET /estatus/123 HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:24:18] "GET /favicon.ico HTTP/1.1" 404 -
```

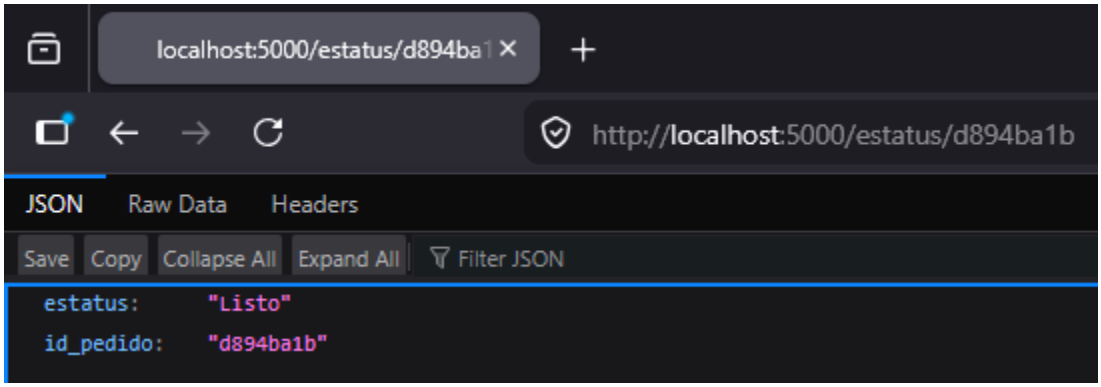
Servicio Web trabajando peticiones



Petición de estatus desde el navegador -> Cliente 1



Petición de estatus desde el navegador -> Cliente 2



Petición de estatus desde el navegador -> Cliente 3

```
127.0.0.1 - - [15/Apr/2026 22:28:05] "GET /estatus/3dc06d29 HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:28:06] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [15/Apr/2026 22:29:20] "GET /estatus/d894bab1 HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:29:20] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [15/Apr/2026 22:29:31] "GET /estatus/d894ba1b HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:29:31] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [15/Apr/2026 22:30:57] "GET /estatus/a0e9e027 HTTP/1.1" 200 -
127.0.0.1 - - [15/Apr/2026 22:30:57] "GET /favicon.ico HTTP/1.1" 404 -
```

Peticiones del navegador recibidas en el Servicio Web

6. Conclusión

La migración a Servicios Web representa un avance significativo en la abstracción del sistema distribuido. Se observó que, aunque la lógica de negocio (cocinar con semáforos) se mantiene idéntica, el medio de transporte HTTP simplifica el desarrollo de clientes y la gestión de la red. La incorporación de métodos GET y POST demuestra cómo los **Servicios Web** permiten gestionar el estado de un sistema distribuido de forma elegante. Se concluye que el uso de identificadores únicos y el almacenamiento de estados en el servidor facilitan la **tolerancia a fallos**: si el cliente pierde la conexión, puede recuperarla simplemente consultando su ID de pedido nuevamente, algo que era imposible con el modelo de sockets tradicional de las prácticas anteriores.